

PDS-FPGAs

PRÁCTICA P1 GRUPO X

Torres Garcia, Antonio
Navarro Sevilla, Juan



UNIVERSITAT
POLITÈCNICA
DE VALENCIA

Índice

1	P1.1: Implementación del módulo DDS	1
1.1	Descripción del módulo	1
1.2	Interfaz	1
1.3	Resultados de implementación	2
1.3.1	Recursos hardware	2
1.3.2	Frecuencia de operación	5
1.4	Verificación	5
1.4.1	Caso 1:	6
1.4.2	Caso 2	7
1.4.3	Caso 3	8
2	P1.2: Análisis de precisión finita de la ruta de datos del modulador de AM	8
3	Conclusiones	9
4	Ficheros entregados	9
5	ANEXO 1: Código HDL del módulo DDS_test	10

1. P1.1: Implementación del módulo DDS

1.1. Descripción del módulo

ENUNCIADO 1

Breve explicación del bloque implementado indicando su funcionalidad y estructura interna. Se puede añadir (de apoyo a la explicación) una figura con el diagrama de bloques interno del modulo.

1.2. Interfaz

ENUNCIADO 2

Incluir las tablas en las que se definan el interfaz, sus formatos y sus parámetros. En esta práctica las tablas están por completar para que sirva de ejemplo de cómo hay que hacerlo.

Tabla 1: Parámetros del módulo DDS_test

NOMBRE	DESCRIPCIÓN
M	Tamaño del acumulador
L	Tamaño tras truncado
W	Tamaño de memoria

Tabla 2: Interfaz del módulo DDS_test

NOMBRE	TIPO	FORMATO	DESCRIPCIÓN
id_p_ac	in	U[M,M]	Paso del acumulador
ic_rst_ac	in	bit	Reset síncrono del acumulador, activo a nivel alto
ic_en_ac	in	bit	Enable del acumulador
ic_data_val	in	bit	Validación de muestras de entrada
clk	in	bit	Entrada de reloj
od_sin_wave	out	S[W,W-1]	Señal senoidal de frecuencia $f_o = P \cdot f_{clk} / 2^M$
od_sqr_wave	out	S[W,W-1]	Señal cuadrada
od_ramp_wave	out	S[W,W-1]	Señal triangular
oc_data_val	out	bit	Señal de variación de salida

1.3. Resultados de implementación

1.3.1. Recursos hardware

ENUNCIADO 3

El objetivo de esta sección es que el alumno sea capaz de identificar si los recursos obtenidos en la implementación son coherentes con los esperables para el circuito realizado. Se deberán incluir las siguientes partes:

- **Estimación de recursos:** Un breve desglose de la estimación de recursos del dispositivo FPGA que se requieren para implementar el módulo, indicando la cantidad de recursos asignados a cada bloque de su diseño. El resultado final se mostrará en una tabla. En esta práctica solo hay que indicar el número de LEs configurados en modo normal, LEs en modo aritmético, los FFs y memorias M9K.
- **Resultados obtenidos:** Una captura del resumen de recursos obtenidos con la herramienta Quartus tras la etapa de emplazamiento y rutado (Fitter), que se muestra en el report "Resource Usage Summary".

- **Análisis del resultado:** Un breve análisis de si los resultados obtenidos son coherentes con la estimación de recursos realizada. El análisis se realizará en base a la comparación entre lo estimado y obtenido en Quartus. Si hay muchas discrepancias con los recursos obtenidos, indicar razonadamente las causas de dicha discrepancia.

En esta primera plantilla de la memoria se ha incluido unas respuestas incorrectas para que sirvan como modelo de cómo se debe contestar. Estas respuestas, así como la captura del report de recursos, deberán ser sustituidas por las de su proyecto.

Estimación de recursos:

Los recursos hardware necesarios para implementar el bloque DDS_test son los siguientes:

- **FFs:** 55 desglosados como 24 del acumulador (bloque b0), 15 del preprocesado (b1) y 16 del post-procesado (b3).
- **LEs normales:** 20 que corresponden al preprocesado (b1).
- **LEs aritméticos:** 40 desglosados en 24 del acumulador (b0) y 16 del post-procesado (b3).
- **M9Ks:** Se requieren 15 para implementar la ROM de $2^{13} \times 16$ bits.

A continuación se muestra la tabla con el resumen de los recursos:

Tabla 3: Recursos hardware

RECURSO	NÚMERO OBTENIDO
FFs	55
LEs-normal	20
LEs-aritméticos	40
M9Ks	15

Resultados obtenidos:

A continuación se muestra la captura realizada de los recursos obtenidos tras la etapa de emplazamiento y rutado (Fitter) en el “Resource Usage Summary”:

Resource	Usage
▼ Total logic elements	82 / 114,480 (< 1 %)
-- Combinational with no register	6
-- Register only	6
-- Combinational with a register	70
▼ Logic element usage by number of LUT inputs	
-- 4 input functions	2
-- 3 input functions	38
-- <=2 input functions	36
-- Register only	6
▼ Logic elements by mode	
-- normal mode	36
-- arithmetic mode	40
▼ Total registers*	76 / 117,053 (< 1 %)
-- Dedicated logic registers	76 / 114,480 (< 1 %)
-- I/O registers	0 / 2,573 (0 %)
Total LABs: partially or completely used	11 / 7,155 (< 1 %)
Virtual pins	0
▼ I/O pins	77 / 529 (15 %)
-- Clock pins	1 / 7 (14 %)
-- Dedicated input pins	0 / 9 (0 %)
M9Ks	17 / 432 (4 %)
Total block memory bits	131,120 / 3,981,312 (3 %)
Total block memory implementation bits	156,672 / 3,981,312 (4 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
PLLs	0 / 4 (0 %)
▼ Global signals	1
-- Global clocks	1 / 20 (5 %)
JTAGs	0 / 1 (0 %)
CRC blocks	0 / 1 (0 %)
ASMI blocks	0 / 1 (0 %)
Oscillator blocks	0 / 1 (0 %)
Impedance control blocks	0 / 4 (0 %)
Average interconnect usage (total/H/V)	0.1% / 0.1% / 0.1%
Peak interconnect usage (total/H/V)	3.2% / 3.7% / 2.5%
Maximum fan-out	93
Highest non-global fan-out	33
Total fan-out	757
Average fan-out	2.23

Análisis del resultado:

Los recursos estimados no son coherentes con los obtenidos en la implementación con Quartus debido a que Quartus realiza optimizaciones y añade lógica adicional no considerada en la estimación teórica, como la inserción de registros y multiplexores para rutear señales y cumplir temporización.

1.3.2. Frecuencia de operación**ENUNCIADO 4**

El objetivo de esta sección es que el alumno sea capaz de identificar el camino crítico del módulo: el que condiciona su frecuencia máxima de funcionamiento.

- *Obtenga la frecuencia máxima de operación del módulo implementado en el dispositivo usando la herramienta TimeQuest Timing Analyzer de Quartus e indique dónde se encuentra el camino crítico del circuito. En la tabla aparece un ejemplo de cómo debe indicar el camino crítico: siempre debe indicar desde qué registro se inicia y en qué registro finaliza.*
- *Realice un breve análisis de si es coherente que el camino crítico sea el obtenido por la herramienta TimeQuest o si cree que debería estar en otra parte del circuito.*

Para realizar el análisis debe tener en cuenta que el camino crítico es el camino entre dos registros con mayor retardo y los retardos en el dispositivo FPGA los producen la lógica combinacional (LUTs) y el rutado. Si un camino tiene más lógica combinacional (más profundidad de lógica: la señal pasa por más LUTs en cascada) generará más retardo debido a la lógica. Si al emplazarse los recursos de un camino concreto (registros y lógica) más separados dentro del chip, esto producirá más retardo por rutado.

Tabla 4: Frecuencia máxima de reloj

f_{clk} (MHz)	CAMINO CRÍTICO
145.65	Desde el registro de la etapa de pre-procesado al registro de la ROM

Análisis del resultado:

Es el que tiene la lógica con más retardo desde un registro al otro.

1.4. Verificación**ENUNCIADO 5**

Resultados de la verificación al excitarlo con diferentes entradas para demostrar su completo funcionamiento. Solo se incluirá:

- *La lista de las pruebas realizadas, que debe coincidir con los casos de test que se*

hayan configurado en el script de simulación de Matlab.

- Las capturas de las formas de onda y el resumen (mostrado en la consola de Modelsim) de la simulación realizada de un máximo de 2 casos, mostrando claramente el comportamiento del circuito y que no existen errores al compararlo con el modelo de Simulink.

Se ha incluido el ejemplo del test número 102. Debe quitarlo y añadir los casos de test que haya realizado.

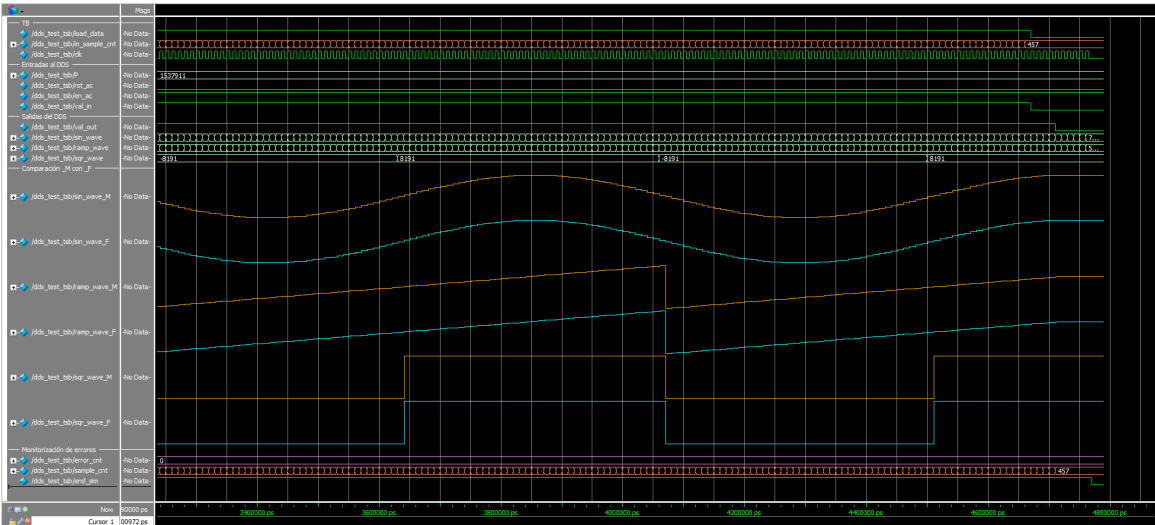
El módulo ha funcionado correctamente. Su funcionamiento se ha verificado con las siguientes pruebas:

1. Test 1: $f_o = 1,1\text{ MHz}$, $M=27\text{ bits}$, $L=15\text{ bits}$, $W=14\text{ bits}$ y $P=1537911.4667$
2. Test 2: $f_o = 40\text{ MHz}$, $M=27\text{ bits}$, $L=15\text{ bits}$, $W=14\text{ bits}$ y $P=55924053.3333$
3. Test 3: $f_o = 0,013\text{ MHz}$, $M=20\text{ bits}$, $L=15\text{ bits}$, $W=14\text{ bits}$ y $P=141.99$

A continuación, se describen dos de los escenarios de test enumerados anteriormente:

1.4.1. Caso 1:

A continuación se muestra el test 1: configuración del DDS con $M=27\text{ bits}$, $L=15\text{ bits}$, $W=14\text{ bits}$ y $P=1537911.4667$, que corresponde a una frecuencia sintetizada de 40 MHz cuando el reloj es de $f_{clk} = 100\text{ MHz}$. Se han simulado 193 muestras y no presenta ningún error entre las muestras simuladas y las del modelo de referencia. A continuación se muestran las diferentes señales y el resumen de la simulación mostrada en la consola de Modelsim.



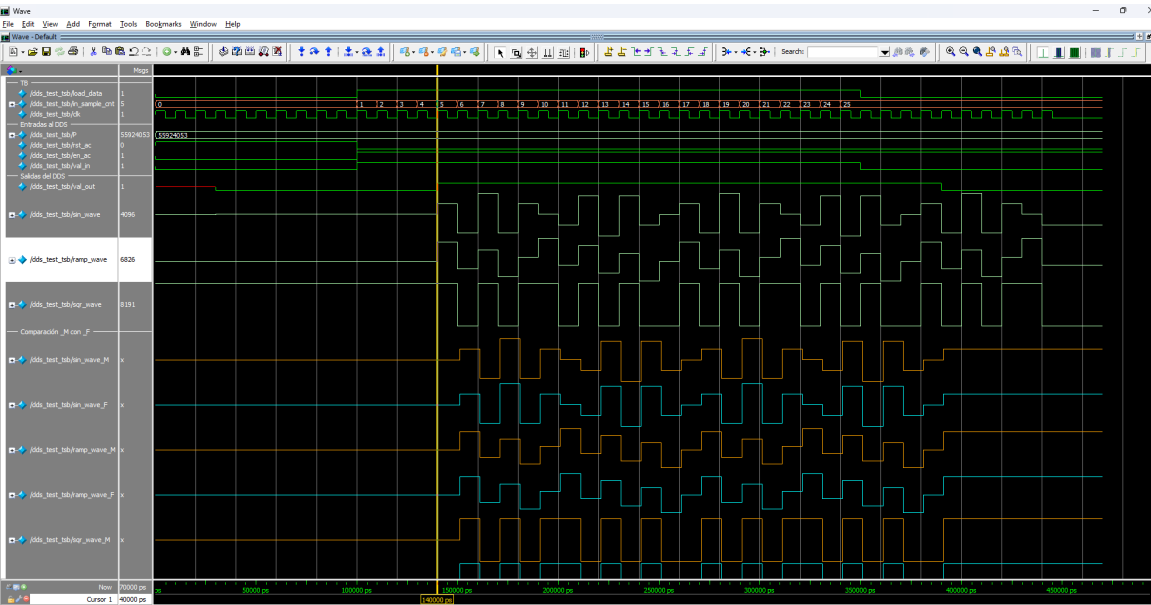
```
*****
** Simulation case 1
*****

fclk = 96 MHz
fo = 1.1 kHz
*****

M = 27 bits
L = 15 bits
W = 14 bits
P = 1537911.4667
*****
```

1.4.2. Caso 2

A continuación se muestra el test 2: configuración del DDS con M = 27 bits, L = 15 bits, W = 14 bits y P = 55924053.3333, que corresponde a una frecuencia sintetizada de 40 MHz cuando el reloj es de $f_{clk} = 100$ MHz. Se han simulado 193 muestras y no presenta ningún error entre las muestras simuladas y las del modelo de referencia. A continuación se muestran las diferentes señales y el resumen de la simulación mostrada en la consola de ModelSim. Se puede observar que las salidas visualizadas en ModelSim no presentan una forma de onda aparentemente coherente; sin embargo, esto no se debe a un error de diseño, sino a que la frecuencia de salida se encuentra muy próxima a la frecuencia de Nyquist, lo que provoca un efecto de submuestreo en la representación temporal.



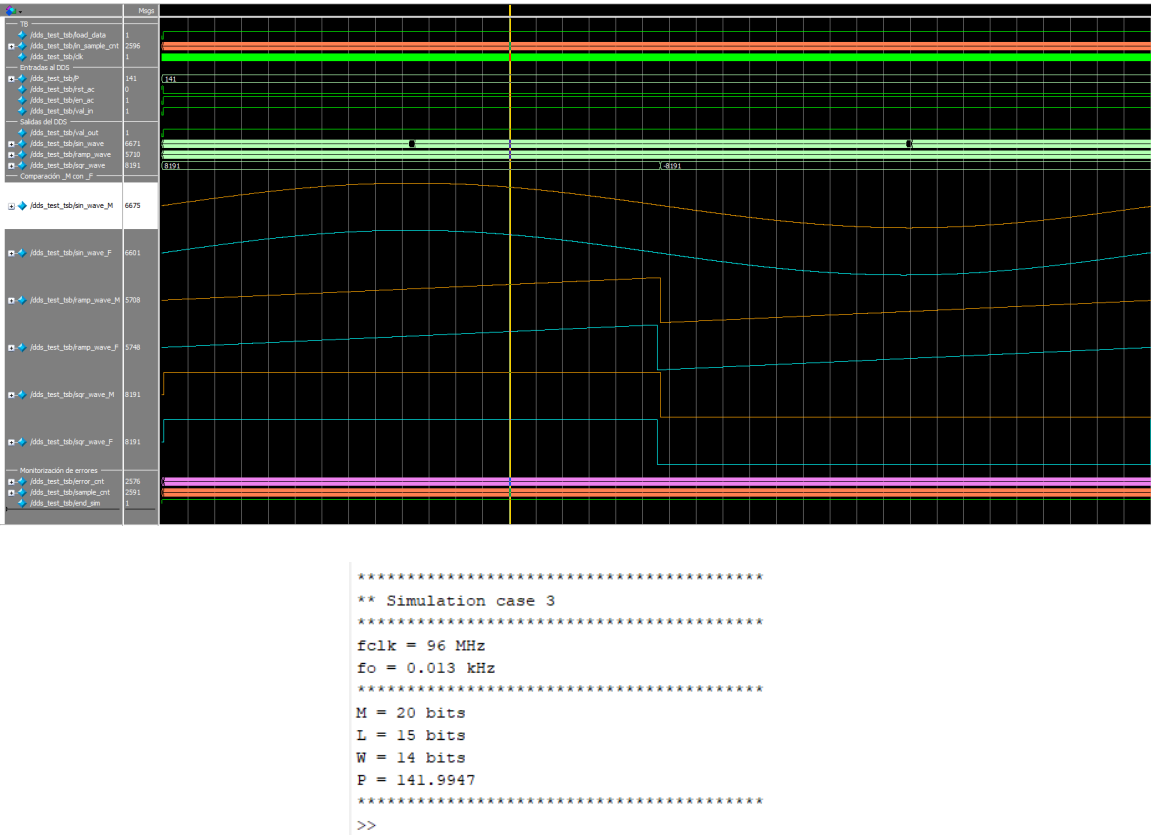
```
*****
** Simulation case 2
*****

fclk = 96 MHz
fo = 40 kHz
*****

M = 27 bits
L = 15 bits
W = 14 bits
P = 55924053.3333
*****
```


1.4.3. Caso 3

A continuación se muestra el test 3: configuración del DDS con $M = 16$ bits, $L = 6$ bits, $W = 16$ bits y $P = 55924053.3333$, que corresponde a una frecuencia sintetizada de 1 MHz cuando el reloj es de $f_{clk} = 100$ MHz. Se han simulado 193 muestras y no presenta ningún error entre las muestras simuladas y las del modelo de referencia. A continuación se muestran las diferentes señales y el resumen de la simulación mostrada en la consola de ModelSim.



2. P1.2: Análisis de precisión finita de la ruta de datos del modulador de AM

ENUNCIADO 6

El modulador con precisión finita debe computar el resultado con al menos una precisión de 12 bits fraccionales. Indique los resultados del análisis de precisión finita al modular con diferentes frecuencias indicando los formatos numéricos y los bits exactos totales obtenidos. Indicar también cuál debe ser el formato numérico a aplicar para que el modulador funcione correctamente en todo el rango de frecuencias.

Tabla 5: Formatos numéricos con diferentes valores de f_{mod} y exactitud en bits

f_{mod}	Q1	Q2	Q3	Q4	Q5	Bits exactos
2 KHz	27	30	29	29	29	12.3
5 KHz	27	30	29	29	29	12.3
10 KHz	27	30	29	29	29	12.36
20 KHz	27	31	30	30	30	12.26

Tabla 6: Formatos numéricos del modulador

Q1	Q2	Q3	Q4	Q5
27	31	30	30	30

3. Conclusiones

ENUNCIADO 7

Conclusiones de la práctica. Valore el grado de cumplimiento de los objetivos de la práctica y el grado de coherencia entre los valores estimados de área y los obtenidos, así como de la frecuencia máxima de funcionamiento.

4. Ficheros entregados

ENUNCIADO 8

Lea el documento [desarrollo y entrega de prácticas](#) e indique cómo se han nombrado los ficheros que se indican en la tabla. En esta práctica esta tabla está por completar, para que sirva de referencia para las siguientes prácticas.

Los autores de la memoria confirman que han entregado los ficheros siguientes, siguiendo las pautas indicadas:

FICHERO	NOMBRE
Memoria de prácticas	Memoria_P1_Gx.pdf
Entrega de los ficheros de prácticas	Proy_sim_P1_Gx.zip
Proyecto Modelsim	dds_test.mpf
Fichero .do	dds_test_tsb.do
Fichero .tcl	dds_test_tsb.tcl
Proyecto Quartus	dds_test.qpf

5. ANEXO 1: Código HDL del módulo DDS_test

ENUNCIADO 9

Incluir un esquema de la división en bloques realizada para codificar el circuito y el código Verilog de los módulos de la práctica. El código debe seguir las pautas indicadas en el documento *desarrollo y entrega de prácticas*.

```
module dds_test
#(parameter M=24, // DDS accumulator wordlength
  parameter L=15, // DDS phase truncation wordlength
  parameter W=16) // DDS ROM wordlength
(
input [M-1:0] id_p_ac, // U[M,0]
input ic_rst_ac,
input ic_en_ac,
input ic_val_data,
input clk,
output signed [W-1:0] od_sqr_wave, // S[W,W-1]
output signed [W-1:0] od_ramp_wave, // S[W,W-1]
output signed [W-1:0] od_sin_wave, // S[W,W-1]
output oc_val_data
);

/* DECLARACIONES ----- */

/*b0 ACUMULADOR*/
logic [M-1:0] b0_ac_r; //S[M:M-1]

/*b1 PRE PROCESADO*/
logic [L-1:0] b1_preProc_s; //U[L:L-1]
logic [L-3:0] b1_preProc_r //U[L-2:L-3]
logic [1:0] b1_ctrlPreProc_s
/* DESCRIPCION ----- */
/*B0*/
always_ff@(posedge clk)
  if(ic_rst_ac)
    b0_ac_r <= 0;
```

```

        else if (ic_en_ac)
            b0_ac_r <= b0_ac_r + id_p_ac;
    /*B1*/
always_comb @(posedge clk)
    begin
        b1_preProc_r
    end
/* ASIGNACION SALIDAS ----- */

assign oc_val_data <= ic_val_data;

assign b1_ctrlPreProc_s = b1_preProc_s[L-1:L-2];
endmodule

/* MEMORIA ROM PARA dds_test */
module dds_test_rom
#(parameter ADDR_WIDTH=13, // Address wordlength
  parameter DATA_WIDTH=14) // ROM output wordlength
(
    input [ADDR_WIDTH-1:0] ic_addr,      // U[ADDR_WIDTH,0]
    input clk,
    output signed [DATA_WIDTH-1:0] od_rom // S[DATA_WIDTH,DATA_WIDTH-1]
);

/* DECLARACIONES ----- */
reg signed [DATA_WIDTH-1:0] rom [0:2**ADDR_WIDTH-1];

logic [DATA_WIDTH-1:0] b0_rom_r;

/* DESCRIPCION ----- */

// ROM data
initial
    if ((DATA_WIDTH == 14)&(ADDR_WIDTH == 13))
        $readmemb("../src/rom_dds_L15_W14.txt", rom);
    else if ((DATA_WIDTH == 16)&(ADDR_WIDTH == 13))
        $readmemb("../src/rom_dds_L15_W16.txt", rom);
    else if ((DATA_WIDTH == 16)&(ADDR_WIDTH == 4))
        $readmemb("../src/rom_dds_L6_W16.txt", rom);

// Read synchronous ROM
always_ff@ (posedge clk)
    b0_rom_r <= rom[ic_addr];

/* ASIGNACION SALIDAS ----- */
assign od_rom = b0_rom_r;

endmodule

```